Least Squares Monte Carlo in Natural Gas Storage Valuation: Report

Answers to the assignment sub-tasks. Figures live in `../results/figures/`, tables in `../results/tables/`.

Note 1: collapsible sections. *Every section and subsection is a drop-down; they start expanded, except the optional-reading blocks (Note 2), which start collapsed (click a heading to collapse or expand it). The drop-downs render in* ***GitHub***, ***GitLab***, *the* ***VS Code Markdown preview*** *(`Ctrl+Shift+V`) and the* ***PyCharm / JetBrains Markdown preview***. *Plain-text viewers, raw editors and some converters (e.g. pandoc to Word/PDF) show the `<details>` tags as text or expand everything instead.*

Note 2: optional reading. *Material that goes beyond what the assignment asks for is marked so it can be skipped. Longer extras sit in a* ***collapsed*** *block titled "**Optional reading (beyond the assignment)**"; short extras that carry a key result stay visible with the tag* ***[beyond the assignment]***. *Everything else answers the assignment directly.*

Note 3: citations. *Papers are cited by short keys, e.g.* ***LS §2.2*** *= section 2.2 of Longstaff & Schwartz (2001).* ***LS***, ***BAW***, ***BdJ*** *and the other keys are listed with full details in* ***5. References*** *at the end. "The paper" means LS in Section 1 and BdJ in Section 4.*

▼ 1. American Options and Least Squares Monte Carlo

Reproduce with `python experiments/01_american_lsmc.py`. Tables in `results/tables/`, figures in `results/figures/`.

▼ Method

At each exercise date the holder compares the immediate payoff `max(K − S, 0)` with the **continuation value** `E[discounted future cash flow | S]`. LSMC (the least-squares Monte Carlo method of Longstaff & Schwartz 2001, [LS]) estimates that conditional expectation by ordinary least squares: regress the realised discounted future cash flow on basis functions of the current spot, using **only ITM paths** (the exercise decision is irrelevant elsewhere; restricting the domain needs fewer basis functions and lowers variance, LS §2.2). I compare immediate exercise against the *fitted* continuation but always discount the *realised* cash flow, never the fitted value, to avoid the upward bias from the convex `max` operator (LS §2.3, fn. 9). Paths are generated with an Euler-Maruyama scheme, as the assignment requires, on a 50-exercise-dates-per-year grid; the valuation uses 100,000 (50,000 + 50,000 antithetic) paths. As a check, every Table 1 point is also priced with the exact log-Euler GBM step on the *same* Brownian increments, which isolates the Euler discretisation bias (see 1a. Table 1 reproduction below).

▼ Choice of basis functions

I use the same basis as Longstaff & Schwartz for their Table 1 (LS §3, using eqs. 2–4): a **constant plus the first three weighted Laguerre polynomials**, evaluated at the spot normalised by the strike, `S/K` (LS §8.3). Why this basis:

- **Polynomial family:** the continuation value is a conditional expectation in L^2, representable as a linear combination of a countable basis (LS §2.2); being linear in the coefficients, fitting is plain, stable OLS.
- **Only three terms:** I use just the fitted value over the in-the-money region, where the put's continuation value is smooth in spot, so a low-order polynomial suffices, more terms don't move much the price (see table below).
- **Laguerre specifically:** orthogonal under the `exp(−x)` weight on `[0, ∞)`. Orthogonality (each coefficient measures something distinct opposed to raw power which move together) keeps the design matrix well-conditioned, unlike raw high powers of `S`.
- **Conditioning:** the `exp(−x/2)` factor underflows on raw prices ~40 (`exp(−20) ≈ 2e-9`), so I feed the basis `S/K` instead (LS §8.3). This keeps the inputs near 1, avoiding the underflow, and makes the fit depend only on moneyness, not the absolute price level or units.

The engine also exposes `powers` and `powers_deg2` (the exact `{1, S, S²}` used to reproduce the paper's 0.1144 worked example).

Does the choice of basis change the price? To check, I priced three Table 1 options again on the **same simulated paths**, changing only the basis (`results/tables/1a_basis_comparison.csv`):

S_0	σ	T	powers_deg2 {1,S,S²}	powers {1,S,S²,S³}	laguerre	paper
36	0.20	1	4.445	4.465	4.464	4.472
40	0.20	1	2.299	2.306	2.306	2.313
44	0.40	2	5.658	5.672	5.669	5.622

All three agree to ~0.01–0.02 (within Monte Carlo error), confirming LS §8.3. Each point is priced on common random (Euler) paths, reusing its Table 1 seed, so the `laguerre` column here is identical to that point's value in `1a_table1_reproduction.csv`. The basis is selected in `configs/section1.yaml` (`common.basis`), so switching is a one-line change with no code edits; the loader rejects unknown names.

▼ 1a. Table 1 reproduction

All 16 tested `(S₀, σ, T)` points, priced on **Euler-Maruyama** paths, match the paper's simulated American values. The **maximum absolute difference is 0.047** (at S_0=44, σ=0.40, T=2; 0.046 at S_0=36, σ=0.40, T=2). I measure it against the combined standard error of the two estimates (combined, because the paper's estimate is noisy too; my estimate and the paper's both have 0.021 there): `√(0.021² + 0.021²) = √0.000882 ≈ 0.0297 ≈ 0.030`, so `0.047 / 0.0297 ≈ 1.58 ≈ 1.6` standard errors. See `results/tables/1a_table1_reproduction.csv`. The stylised 8-path example reproduces **0.1144** exactly (unit test).

▼ *Optional reading (beyond the assignment): how much of the gap is Euler bias*

How much of that is Euler bias? The table also prices every point with the exact log-Euler GBM step on the *same* Brownian increments (`exact_American`); because the noise is shared, `euler_minus_exact` is the pure Euler discretisation bias. It is **positive** (Euler overprices) and grows with volatility and maturity: **≤0.004 at σ=0.20** (below half a standard error) and **0.012–0.019 at σ=0.40** (up to ≈0.9 of a standard error). This is why the σ=0.40 differences to the paper lean positive. On the exact paths the maximum difference to the paper is 0.034, so the scheme choice does not change the conclusion; it shows that at 50 steps per year Euler's bias is visible only at high volatility and stays inside Monte Carlo error. The step-size dependence of this bias is studied in 1b.

▼ 1b. Discretisation step (1 day → 1 week)

To isolate the Euler bias from Monte Carlo noise I use a **nested-grid** design: draw one fine Brownian path, then build each coarser grid by summing consecutive fine increments, so every step size (and its exact reference) rides the *same* underlying Brownian motion. The noise then cancels across the whole curve, not just point by point. The resulting bias **grows with both step size and volatility**. At σ=0.40 it rises from ≈0.001 at a one-day step to ≈0.022 at a one-week step; at σ=0.25 it stays within a few thousandths (≤0.006); at σ=0.10 it is negligible (≤0.001), i.e. Euler is essentially unbiased at low volatility. See `results/figures/1b_discretisation_bias.png` and `results/tables/1b_discretisation_step_study.csv`.

▼ 1c. Heston stochastic volatility (μ=0, ρ>0 and Feller `2κθ > ξ²` per the assignment)

What the regression uses: spot and variance. Under Heston [H] the option's value today depends on two things, the spot `S` and the current variance `v`, so the regression uses both. LS suggest this for two-variable problems: use powers of each variable plus their product (LS p. 123, "terms in X and in Y, as well as cross-products"). My basis is `{1, S, S², v, v², S·v}`.

I use plain powers here, not Laguerre polynomials: Laguerre is built for one positive variable and does not carry over well to two variables of very different size. For the same reason I rescale both before fitting, `S` by the strike and `v` by θ, so each is around 1. Otherwise `v ≈ 0.006` would sit next to `S ≈ 1` and the least-squares fit would be numerically unstable.

▼ *Optional reading (beyond the assignment): does the second regression factor matter?*

Does the second factor matter? I priced both ways on common random paths at three moneyness levels (`results/tables/1c_heston_basis_comparison.csv`). In this regime the difference is ≈0 everywhere: with `r = 0` and the low variance level fixed by the assignment, the American put has a very small early-exercise premium, so the exercise boundary, the only thing the regression influences, barely depends on variance. The `(S, v)` basis is the theoretically correct choice and becomes material when the early-exercise premium is larger (higher rates or deeper in-the-money); spot-only is a good, cheaper approximation here.

Parameter sensitivities across moneyness. I vary one parameter at a time, keeping the others at the base case, and price the put at $S_0 \in \{0.85$ (ITM), 1.00 (ATM), 1.15 (OTM)$\}$, K=1. Because the three price levels differ ~50× (ITM ≈0.15 vs OTM ≈0.006), the figure plots the **relative change price/price(base)**, so every line passes through 1.0 at the base value (each range of values includes it) and the slopes are directly comparable. Base case: $v_0=0.08^2$ (fixed by the assignment), **$\kappa=5.0$**, **$\theta=0.12^2 > v_0$** (so κ has a gap to act on), ξ=0.10, **$\rho=+0.3$** (positive, as the assignment requires; commodities show the inverse of the equity leverage effect). The base κ is set to 5.0 so the Feller bound $2\kappa\theta=0.144$ leaves room to test ξ up to 0.35; Feller holds at every value I test. See `results/figures/1c_heston_sensitivities.png` and `results/tables/1c_heston_sensitivities.csv`.

- **θ (long-run variance):** raises the put at every moneyness; the *relative* effect is by far the largest OTM (S_0=1.15: rel-to-base 0.19 → 2.39 from the lowest to the highest θ, i.e. price 0.0011 → 0.0139) vs ATM ×1.3 and ITM only +4%. More variance most helps options that need a large move to pay off.
- **κ (mean-reversion speed):** clearly positive because $\theta > v_0$: faster reversion pulls variance up toward the higher long-run level sooner. Again strongest OTM (rel 0.34 → 1.11, price 0.0020 → 0.0065) and material ATM (0.78 → 1.02), while ITM is nearly flat (already deep enough that the extra variance barely matters).
- **ξ (vol of variance):** fattens the variance tails, but the net price effect is small at r=0: it lifts deep-ITM slightly (rel 1.00 → 1.01) while pulling ATM (1.00 → 0.98) and OTM (1.04 → 0.87) *down*. Not a clean "raises the wings" signature;
- **ρ (spot-vol correlation, positive): **lowers OTM puts** (rel 1.11 → 0.80 across ρ = 0 → 0.9, price 0.0064 → 0.0046) while barely lifting ITM and leaving ATM flat. Positive correlation means low prices coincide with low volatility, thinning the left tail and so cheapening deep-OTM puts, the clearest skew effect, and one the ATM-only view misses entirely.

▼ 2. Control Variates for American Options

Reproduce with `python experiments/02_control_variates.py`. Tables in `results/tables/`, figures in `results/figures/`. One-day time steps throughout (`n_steps = round(T × 252)`).

▼ Method

A **control variate** reduces Monte Carlo variance by subtracting a correlated quantity whose mean is known exactly. The European put is ideal: it shares the American put's terminal spot (so their payoffs are highly correlated) and its mean is the closed-form Black-Scholes value. On one shared set of GBM paths I form, per path, the discounted American PV `X` (LSMC) and the discounted European payoff `Y = e^{-rT} max(K - S_t, 0)`, then combine

`X_cv = X - β (Y - E[Y])`, `β = cov(X, Y) / var(Y)`, `E[Y] = bs_put(...)`.

Since `E[Y]` is exact, `E[X_cv] = E[X]` (unbiased); the optimal β minimises the estimator variance, which falls by the factor `1/(1 - ρ²)` in the ideal case.

The r = 0 subtlety (assignment mandate). Section 2 sets *all* rates to zero. Theoretically, when `r = 0` it is never worth exercising a put early, so the American put is just the European put (confirmed in `results/tables/appendix/2d_r0_american_equals_european.csv`). This also makes BAW's `k = 1 - e^{-rT} = 0`, a `0/0` I handle with an explicit r = 0 guard that returns the European value.

▼ *Optional reading (beyond the assignment): why I also run r = 0.06*

I also run `r = 0.06` (the Table-1 rate). Not because the control variate does better there, it does better at r = 0, but because r = 0 is a degenerate test: the put isn't really American. At r = 0.06 there is a genuine early-exercise premium, so the method is tested on a true American option. Volatility is swept in both cases; only interest/carry rates are zeroed.

▼ 2a. European-put control variate

At-the-money one-year put, r = 0 as the assignment requires, vol swept 0.10 → 0.50 (`results/tables/2a_cv_correlation.csv`, `results/figures/2a_cv_variance_reduction.png`).

σ	corr(Amer, Euro)	variance reduction
0.10	0.874	4.2×
0.30	0.907	5.7×
0.50	0.915	6.2×

The control variate cuts the estimator variance **4–6×**, and the gain grows with volatility (at higher volatility both options' payoffs depend more on where the spot ends up, which the European put captures, so the two move together more closely). *Why the correlation is not 1.0, even though at r = 0 the American and European prices match: LSMC's stopping rule is a noisy regression estimate, not the true policy, so a few paths exercise early by mistake, and those decorrelate the American PV from the European payoff.*

▼ *Optional reading (beyond the assignment): the same test at r = 0.06*

σ	corr(Amer, Euro)	variance reduction
0.10	0.582	1.5×
0.30	0.777	2.5×
0.50	0.824	3.1×

At r = 0.06 the reduction is smaller (**1.5–3×**) and the correlation lower (0.58–0.82), because the early-exercise premium is exactly the part of the American payoff the European control cannot track, so the reduction is real but smaller. The same figure and table hold both rates.

▼ 2b. Barone-Adesi-Whaley benchmark

Barone-Adesi & Whaley (1987) [BAW] is a fast analytic American approximation used here as the reference value. For one-month puts at σ ∈ {0.05, 0.10, 0.20, 0.30} I compare `|LSMC − BAW|` with and without the control variate (`results/tables/2b_baw_error.csv`, `results/figures/2b_cv_baw_error.png`). At r = 0 BAW reduces to the European value; ***[beyond the assignment]*** at r = 0.06 it carries a real premium (e.g. σ = 0.30: BAW 1.2923 vs European).

Both estimators already sit within **≈0.001–0.005** of BAW at these path counts, so the error-vs-BAW is dominated by the residual **bias** (LSMC's low-bias sub-optimal stopping and the finite time grid; BAW's own approximation), not by estimator variance. The control variate attacks variance, not bias, so the two error curves are similar and their tiny difference is not consistently signed;

▼ 2c. Gamma profile

Gamma, $\partial^2 V/\partial S^2$, measures the **curvature** of the option value in spot, how fast the delta (hedge ratio) changes as the spot moves. I estimate it by a **central finite difference**: price the put at three spots (S_0-h, S_0, S_0+h) and form $[V(S_0+h) - 2V(S_0) + V(S_0-h)] / h^2$. A second difference of noisy Monte Carlo prices is very unstable, so I use **common random numbers**, the same Brownian path at all three spots, which cancels the shared noise and leaves the curvature. Repeating over independent batches gives the estimator variance the assignment asks for (`results/tables/2c_gamma_profile.csv`, `results/figures/2c_gamma_profile.png`; one-month put, r = 0).

The profile is **positive everywhere and peaks at the strike**: gamma at K = 40 is ≈0.34 at σ = 0.10, falling to ≈0.11 at σ = 0.30. This is the expected shape, a put is most curved at the money (where it is most uncertain whether it finishes in or out), and nearly linear far from the strike; higher volatility spreads that uncertainty over a wider range of spots, so the peak flattens. The common-random- numbers trick keeps the estimate tight near the money (standard errors ≈0.002–0.007); the estimator variance is largest around the peak, where the value surface bends most.

▼ 3. Convergence of the Stochastic Differential Equation

Reproduce with `python experiments/03_sde_convergence.py`. Tables in `results/tables/`, figures in `results/figures/`.

▼ 3a. Bias-variance decomposition of the error

The error of an LSMC price splits into two independent parts:

`total error² ≈ bias² + variance`

- **Variance**, Monte Carlo sampling noise from using finitely many paths. It is *random* (a different seed gives a different estimate) and shrinks like `1/√(n_paths)`. This is what Section 2's control variate attacked.
- **Bias**, a *systematic* offset that does **not** average out with more paths. Here it is the **time-discretisation error**: I solve the SDE on a finite grid, so even the exact expectation of my discretised paths differs from the true continuous-time price. It is set by the step size and the *scheme*, not the path count.

The practical consequence: adding paths drives variance down but leaves a **bias floor**. `results/tables/3a_bias_variance_decomposition.csv` and `results/figures/3a_bias_variance_decomposition.png` show this directly, on a fixed 50-step Euler grid, as paths grow 10k→100k the standard deviation falls (0.034 → 0.009) while the bias stays put (≈0.015–0.027), so the RMSE flattens onto the bias floor rather than reaching zero. To reduce that floor you must change the *scheme*, which is what the Milstein method does.

▼ 3b. Milstein scheme: GBM

The Milstein (1975) scheme [M] adds the Itô second-order term from the state-dependent diffusion, for GBM, `0.5 σ²S (ΔW² − Δt)`, raising the strong convergence order from 0.5 (Euler) to 1.0. On a coarse 25-step grid I price with each scheme across 10k→100k paths and measure the error against **two** references (`results/tables/3b_gbm_scheme_convergence.csv`, `results/figures/3b_gbm_scheme_convergence.png`):

- **vs BAW** (the comparison the assignment asks for): Euler looks *closer* to BAW than Milstein. This is a reference artefact, not a real win: BAW is itself only an approximation and sits ≈0.02 above the true price, so it happens to flatter Euler's upward bias.
- ***[beyond the assignment]*** **vs the exact GBM** (log-Euler on the *same* Brownian paths, common random numbers, so the comparison is noise-free): this isolates pure discretisation bias. **Milstein's bias is ≈−0.012 and stable; Euler's is ≈+0.02 to +0.03**, roughly half, as the order improvement predicts. I add it because BAW's own error hides the result above; this is a more fair comparison.

▼ 3c. Milstein scheme: Heston

BAW cannot be used under Heston: it assumes constant volatility (GBM), and there is no BAW-style formula for stochastic volatility. The reference is therefore a high-resolution 400-step simulation; I compare coarse 10-step Euler and Milstein against it across the 10k→100k path range (`results/tables/3c_heston_scheme_convergence.csv`, `results/figures/3c_heston_scheme_convergence.png`). My simulator uses **exact log-Euler on the spot**, so only the *variance* leg is discretised, and the Milstein correction there, `0.25 ξ² (ΔW² − Δt)`, is proportional to ξ^2, so it only makes a real difference when the **vol of variance ξ is large** and the variance moves a lot from step to step.

Building a clean reference. For the gap "coarse − reference" to be pure discretisation bias, the two runs may differ *only* in step size. I therefore use the same nested-grid design as 1b and 3b. At each path count I draw one set of correlated 400-step increments `(ΔWᵛ, ΔWˢ)`; the 10-step increments are the sums of each block of 40, so both grids ride the **same Brownian motion** with the **same number of paths**. The reference is simulated on the fine grid but may only be **exercised on the 10 coarse dates**; otherwise it would also contain the value of being able to exercise on more dates (not fair comparison)

▼ *Optional reading (beyond the assignment): how large the exercise-date effect is*

An earlier version priced the reference on independent paths and let it exercise on every fine step. Its gap mixed Monte Carlo noise and this extra exercise value into the bias. The table keeps `ref_all_dates`, the every-step reference, to show the size of that effect: about +0.0004 in base and −0.002 in stress. In base it is about twice the bias itself.

I show two regimes (the assignment fixes only v_0, $\rho>0$, $\mu=0$ and $2\kappa\theta>\xi^2$, not ξ, so both ξ values are my choice):

- **base** (ξ = 0.10, my Section-1 base case): the variance leg is smooth, so Euler and Milstein are indistinguishable. Both biases are ≈−0.0002 (−0.0001 to −0.0003 across path counts, about 1% of the 0.021 price), and the two schemes agree to within 0.00002 at every path count. Milstein adds nothing here.
- ***[beyond the assignment]*** **stress** (ξ = 0.60): ξ is large, so the variance moves a lot each step, and Milstein cuts the bias from **≈0.0023 to ≈0.0007 (≈70%)**. Milstein is closer to the reference at every path count (Euler 0.0017–0.0026, Milstein 0.0001–0.0011).

Because the noise is shared, the bias is stable as paths go from 10k to 100k: a genuine bias floor that more paths cannot remove; only the better scheme does. The small residual wobble from row to row is the LSMC regression noise. The coarse and fine runs fit their own regressions, and that noise does not fully cancel.

▼ 3d. Why Milstein improves convergence

Euler-Maruyama holds the diffusion coefficient $\sigma(S)$ constant across each step, ignoring that it changes *with* the state as the path moves within the step. For a state-dependent diffusion (GBM's σS, the variance leg's $\xi\sqrt{v}$) that omission is an $O(\Delta t)$ error per step. Milstein adds the leading Itô correction for exactly this term ($\frac{1}{2}\,\sigma\sigma'\,(\Delta W^2 - \Delta t)$), capturing the within-step variation of the coefficient. This raises the strong order from 0.5 to 1.0, the paths themselves are closer to the true SDE solution, so the discretisation **bias** shrinks and the estimator converges to a value nearer the truth for the same step size. It does nothing for variance (that still needs more paths or a control variate), the two levers act on the two independent error terms.

▼ 4. Usage in Natural Gas Storage

Reproduce with `python experiments/04_gas_storage.py`. Tables in `results/tables/`, figures in `results/figures/`. Following Boogert & de Jong (2008) [BdJ], "the paper" in this section.

▼ Method

A storage contract generalises the American option: instead of one exercise, the holder chooses **inject, hold or withdraw** every day, subject to a working capacity and daily injection/withdrawal rates. This makes the state two dimensional (spot price *and* current inventory), so the problem is a dynamic program over a time by inventory grid (paper eqs. 10 to 12). I solve it by backward induction: at each day and inventory node, pick the action maximising immediate cash flow $h(S, \Delta v) = -S \cdot \Delta v$ (inject costs, withdraw earns; rates and spreads are zero) plus the continuation value of the resulting inventory, where the continuation value is estimated by regressing realised future cash flows on B-splines of the current spot. I use the paper's **b6** basis exactly (footnote 4: order-4 cubic B-splines, `nbasis = 6`, i.e. 2 evenly spaced interior knots over each day's price range); no separate constant is needed, since B-splines sum to 1. Unlike Section 1 I regress on **all** paths, not just in-the-money ones: storage has negative cash flows (injections), so "in the money" is undefined (paper footnote 3).

Three design points keep this faithful and efficient:

1. **Inventory grid aligned to the rates.** With the paper's discretisation (α = 2500 MWh, `N = 101`; "Standard Storage Contract", p. 89) I have `max_inject` = α and `max_withdraw` = 3α, so every allowed action moves a whole number of grid steps and reachable inventories land exactly on nodes. No interpolation (paper eqs. 26 to 28) is needed. The step α is a user setting in `configs/section4.yaml` (`storage.alpha`); any value that divides the capacity, volumes and rates works, and off-grid values are rejected.
2. **Dimension reduction (eq. 16).** Continuation depends only on the post-decision inventory, and the regression design matrix depends only on spot, so all inventory-node regressions at a time step share it and solve in one least-squares call.
3. **Decision versus realised value.** The fitted continuation chooses the action, but I accumulate the realised future cash flow (the same anti-bias rule as Section 1). A terminal penalty forces the strategy back to the required end inventory.

▼ 4a. Day-ahead price simulation

The spot price follows the one-factor Schwartz [S] mean-reverting log-price process (BdJ eq. 22), stepped daily over the one-year contract (`results/figures/4a_price_simulation.png`, `results/tables/4a_price_paths_summary.csv`). The assignment lets me pick μ as a constant (μ = 3), so prices revert to a flat level rather than tracking a seasonal forward curve. Paths start at $\exp(\mu) \approx 20.1$ EUR/MWh and settle around a long-run mean of **≈ 19.2** EUR/MWh. The long-run mean is below $\exp(\mu)$ because the log-price reverts to $\mu - \sigma^2/(2\kappa) \approx 2.91$ (BdJ eq. 23), and the spread of the lognormal adds back only part of that, giving $\exp(\mu - \sigma^2/(4\kappa)) \approx 19.2$. In the figure the dashed line marks this long-run mean and the dotted line the start level $\exp(\mu)$. The cross-path mean (red) drifts from 20.1 down to the dashed line within the first month and stays there (simulated mean terminal price 19.22). The 90% band (5th to 95th percentile) widens over the first month and then stays at about 11 to 30 EUR/MWh for the rest of the year, and the price distribution is right-skewed (lognormal, as a log-OU process gives). **Caveat:** with a flat μ there is almost no seasonal spread, so the *intrinsic* value is near zero and what I price is essentially the *extrinsic* value (the worth of reacting to volatility). This is why my figure differs from the paper, which used a real seasonal TTF forward curve.

▼ 4b. Storage valuation

Valuing the Exhibit-1 contract (250,000 MWh capacity, 100,000 MWh start and end) on 10,000 paths gives **≈ EUR 4.65 million** (standard error ≈ EUR 11k), or about 18.6 EUR per MWh of capacity (`results/tables/4b_storage_value.csv`). This is the same order of magnitude as the paper's EUR 5.4 million high-volatility figure; the gap is expected, since my flat `μ` omits the seasonal intrinsic value their forward curve carries. The value is driven here by the flexibility to buy on low-price days and sell on high-price days.

▼ 4c. Convergence and value distribution

I value the contract at several path counts, 10 runs each (`results/tables/4c_convergence.csv`, `results/figures/4c_convergence.png`). The value stabilises quickly: EUR 5.30m at just 50 paths, then EUR 4.69m, 4.66m and 4.65m at 500, 5,000 and 10,000 paths, with the across-run standard deviation shrinking from EUR 173k (50 paths) to EUR 13k (10,000 paths). This reproduces the paper's striking observation that storage valuation needs far fewer paths than a plain American option (they attribute it to strong mean reversion keeping the price distribution narrow, plus the dimension reduction). The 50-path value sits noticeably **high**. This is possiblt the in-sample LSMC optimism (with very few paths the regression overfits the paths it is trained on, inflating the fitted continuation and the value); it disappears by 500 paths.

The right-hand panel is the Exhibit-3 plot: as in the paper, it is the distribution of the realised value on each individual path from a single valuation (the 4b run, 10,000 paths), and the storage value is its mean (EUR 4.65m). The distribution is bell-shaped and roughly symmetric, ranging from about EUR 1m to EUR 8m. This wide per-path spread is the risk of operating the storage on any one price scenario; the precision of the *value* is the much smaller standard error of the mean (≈ EUR 11k), as in 4b.

▼ 5. References

The papers in `assignment/` are the ones the assignment names. Keys in square brackets are how the report cites them; `§` is a section and `eq.` an equation of that paper.

Key	Reference	Used in	Copy in repo
[LS]	Longstaff, F. A. and Schwartz, E. S. (2001). "Valuing American Options by Simulation: A Simple Least-Squares Approach." *The Review of Financial Studies*, 14(1), 113–147.	§1 (LSMC algorithm, basis, Table 1), §2–3 (pricer)	`assignment/LongstaffSchwartzAmerican`
[BAW]	Barone-Adesi, G. and Whaley, R. E. (1987). "Efficient Analytic Approximation of American Option Values." *The Journal of Finance*, 42(2), 301–320.	§2b, §3b (analytic American benchmark)	`assignment/Barone_Adesi_Whaley_1987.`
[M]	Milstein, G. N. (1975). "Approximate Integration of Stochastic Differential Equations." *Theory of Probability and Its Applications*, 19(3), 557–562.	§3 (Milstein scheme)	`assignment/1119062.pdf`
[BdJ]	Boogert, A. and de Jong, C. (2008). "Gas Storage Valuation Using a Monte Carlo Method." *The Journal of Derivatives*, 15(3), 81–98.	§4 (storage model, eqs. 10–16 and 22, b6 basis, Exhibits 1, 3 and 6)	`assignment/JOD-Gas-storage-valuation`

Key	Reference	Used in	Copy in repo
[H]	Heston, S. L. (1993). "A Closed-Form Solution for Options with Stochastic Volatility with Applications to Bond and Currency Options." *The Review of Financial Studies*, 6(2), 327–343.	§1, §3c (stochastic-volatility model; the assignment gives its SDEs)	not in repo
[S]	Schwartz, E. S. (1997). "The Stochastic Behavior of Commodity Prices: Implications for Valuation and Hedging." *The Journal of Finance*, 52(3), 923–973.	§4a (one-factor mean-reverting price model, BdJ eq. 22)	not in repo

Section pointers for LS (the numbers cited in the text): §2.2 the LSM algorithm and basis functions (eqs. 2–4, Laguerre); §2.3 convergence; §3 the American put example and Table 1; §8.3 choice of basis functions (including the division by the strike).